



Версії схеми та структури бази даних

Огляд

Відстеження бази даних вашого додатка – завдання не з легких. Схеми бази даних, як правило, не збігаються в різних середовищах, дані в одній із баз даних можуть пропустити певну важливу частину даних. Такі випадки можуть викликати роздратування, особливо коли їх спіймають на виробництві.

Ситуація погіршується, коли ви розробляєте програмне забезпечення, що розповсюджується. У цьому випадку кожен із ваших клієнтів має власний екземпляр бази даних, структура якого може відрізнятися від інших. У такому проекті відстеження баз даних ваших клієнтів може стати кошмаром.

Давайте розглянемо найкращі методи керування версіями бази даних, які допоможуть нам вирішити цю проблему.

1. Керування версіями бази даних: проблема

Коли ви самотній програміст, який працює над проектом, який ще не відправлений у виробництво, немає такої проблеми, як версії бази даних. Ви просто змінюєте свою схему БД так, як хочете, і вона завжди працює.

Проблеми виникають, коли ваше програмне забезпечення починає працювати у виробництві або до вас приєднується новий член команди для роботи над частинами вашого проекту, пов'язаними з базою даних. Щойно у вас буде більше одного екземпляра бази даних, вони починають розсинхронізуватися. Навіть з одним екземпляром потрібна значна кількість часу для синхронізації змін, якщо з ним працює більше ніж один розробник.

2. Найкращі практики керування версіями бази даних

На щастя, ми не одні. На цю тему написано багато матеріалів, а також програмного забезпечення, яке покликане вирішити цю проблему. Це остаточне керівництво щодо того, як розвивати вашу базу даних разом із кодом, який її використовує.

Гаразд, то які найкращі практики керування версіями бази даних?

Найкраща практика #1: нам потрібно розглядати базу даних програми та довідкові дані в ній як звичайний код. Це означає, що ми повинні зберігати як його схему, так і довідкові дані в системі керування джерелами.

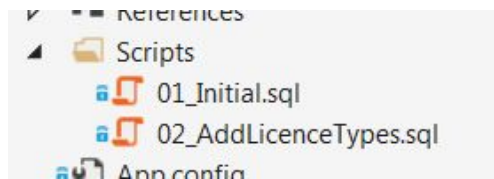
Зауважте, що це правило включає не лише схему бази даних, а й довідкові дані в ній. Довідкові дані - це дані, які є обов'язковими для запуску програми. Наприклад, якщо у вас є словник усіх можливих типів клієнтів, існування яких залежить від вашої програми, ви також повинні зберігати його в системі керування джерелами.

2. Найкращі практики керування версіями бази даних

Найкраща практика #2: ми повинні явно зберігати кожну зміну в схемі бази даних і в довідкових даних. Це означає, що для кожної зміни, яку ми вносимо, ми повинні створити окремий сценарій SQL зі змінами. Якщо модифікація впливає як на схему, так і на довідкові дані, їх слід відобразити в одному сценарії.

Дотримання цього правила є важливою частиною побудови успішної системи контролю версій бази даних. У багатьох проектах схема бази даних зберігається в системі керування джерелами, але часто це просто знімок останньої версії бази даних, якою б вона не була. Усі зміни в ньому відстежуються самою системою керування джерелами, вони не зберігаються явно. Більш того, часто зміни в довідкових даних взагалі не відстежуються.

Такі інструменти, як проект бази даних Visual Studio, підкреслюють цей підхід і закликають програмістів використовувати автоматично створені сценарії оновлення для оновлення схеми. Хоча це може добре працювати в малих проектах, у великих проектах відстеження змін у базі даних за допомогою автоматично створених сценаріїв стає тягарем.



2. Найкращі практики керування версіями бази даних

Найкраща практика #3: кожен файл сценарію SQL має бути незмінним після його розгортання у робочому або проміжному середовищі.

Суть збереження змін в окремих файлах полягає в тому, щоб мати можливість відстежувати кожен з них. Коли ми змінюємо існуючі сценарії SQL, ми втрачаємо всі переваги, які надає нам найкраща практика управління версіями бази даних. Зберігайте файли сценаріїв незмінними після їх розгортання. Якщо вам потрібно відхилити зміни, які вже надіслано, створіть для цього окремий сценарій.

Найкраща практика #4: всі зміни в схемі бази даних і довідкових даних повинні застосовуватися через скрипти. Жоден з них не можна застосувати вручну.

Якщо ми змінюємо базу даних, передаючи наші сценарії, вся ідея керування версіями бази даних стає марною, тому нам потрібно переконатися, що зміни вносяться лише за допомогою сценаріїв SQL, які ми створюємо.

2. Найкращі практики керування версіями бази даних

Найкраща практика #5: кожен розробник у команді повинен мати власний екземпляр бази даних.

Часто команди починають з однієї бази даних у середовищі розробника. На початку це добре працює, але коли база даних стає достатньо великою, її одночасні модифікації стають усе важчими й важчими, поки в якийсь момент не перестануть працювати взагалі.

Зміни, які вносять програмісти, часто несумісні, тому кожному програмісту доцільно мати окремий екземпляр БД, щоб уникнути таких колізій. Якщо розробники одночасно змінюють пов'язані фрагменти схеми БД, такі конфлікти можна вирішити за допомогою системи керування джерелами, як і конфлікти в кодї C#/Java/etc.

Крім того, якщо у вас є кілька гілок вашої кодової бази, ви також можете створити окремий екземпляр БД для кожної з них, залежно від того, наскільки різні бази даних у цих гілках.

2. Найкращі практики керування версіями бази даних

Найкраща практика #6: версія бази даних повинна зберігатися в самій базі даних. Я зазвичай створюю окрему таблицю під назвою «Параметри» і зберігаю там версію. Не використовуйте складні позначення, як-от "x.y.z", для номера версії, використовуйте лише одне ціле число.

3. Які переваги такого підходу?

Отже, які переваги дають нам ці найкращі методи управління версіями бази даних?

Перша і найважливіша перевага полягає в тому, що коли ми використовуємо цей підхід, ми більше не маємо проблем із невідповідністю схем бази даних. Автоматичне оновлення до останньої версії повністю вирішує їх, звичайно, якщо ми повністю дотримуємося правил, описаних вище.

Це особливо корисно, коли у вас немає єдиної робочої бази даних, але кожен клієнт має власний екземпляр БД. Управління версіями БД за таких обставин може стати пеклом, якщо ви не використовуєте належні методи керування версіями.

Ще одна перевага, яку забезпечують ці найкращі практики, — висока зв'язність змін бази даних. Це означає, що кожна помітна зміна в схемі та довідкових даних відображається в одному місці, а не поширюється на програму.

3. Які переваги такого підходу?

Сценарії оновлення SQL також мають високу когезію, оскільки вони містять усі зміни БД, необхідні для функції, тому легко зрозуміти, які зміни було внесено в базу даних, щоб розблокувати певну функціональність. Зберігання змін схеми та даних, пов'язаних одне з одним, в одному файлі також дуже допомагає.

Підхід, описаний у цій публікації, застосовний, навіть якщо ви не дотримувалися його з самого початку. Щоб застосувати це на практиці, вам просто потрібно створити початковий сценарій зі схемою бази даних, яку ви зараз маєте у виробництві, і почати поступово змінювати її з цього моменту. Поточна версія має стати версією №1, від якої ви можете переходити далі, використовуючи прийоми, які ми обговорювали вище.

Питання та відповіді